

RECORD

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	5
Student Name	M. Naga Doleswari Aruna Devi
Roll No.	A24126552267
Date	30-08-2026

1. TITLE

Working with Node.js Modules - ES Modules, CommonJS, and the Built-in os and fs Modules

2. AIM

To create and use custom modules in Node.js with both ES module (import/export) and CommonJS (require/module.exports) syntax, and to use Node's built-in os and fs modules to read system information and perform file operations.

3. OBJECTIVE

The objectives of this experiment are:

- To create a custom ES module (message.js) that exports a function.
- To import and use an exported function in another file using import.
- To use Node's built-in os module to retrieve system information.
- To use Node's built-in fs module to write to and read from a file synchronously.
- To create a custom CommonJS module (calculator.js) that exports multiple functions.
- To import and use a CommonJS module in another file using require().

4. THEORY

Node.js supports two module systems. CommonJS is Node's original system, where a module exports values through module.exports and another file loads it with require(). ES Modules are the standard JavaScript module system, where a module exports values with the export keyword and another file loads them with import; enabling this syntax in a plain .js file requires "type": "module" to be set in package.json (or the file to use a .mjs extension).

Node also ships a number of built-in modules that do not need to be installed. The os module provides information about the operating system, such as os.type(), os.platform(), os.arch() and os.homedir(). The fs module provides file system operations; fs.writeFileSync() and fs.readFileSync() perform synchronous writes and reads, meaning the program waits for each operation to finish before moving to the next line.

A custom module groups related functions together so they can be reused across files. calculator.js defines multiply() and divide() and exposes them through module.exports, so app.js can load the whole module with require('./calculator') and call its functions through the returned object.

5. ALGORITHM / PROCEDURE

- Enable ES module syntax for the project (set "type": "module" in package.json, or use a .mjs extension).
- Create message.js and define a welcome(name) function that returns a greeting string, exported using export.
- Create main.js, import welcome from message.js, and call it with a sample name.
- Run main.js using Node.js and verify the greeting is printed to the console.
- Create system.js and load the built-in os module using require().
- Use os.type(), os.platform(), os.arch() and os.homedir() to print system details.
- Run system.js and verify the system information is printed correctly.
- Create a file-handling script and load the built-in fs module using require().
- Use fs.writeFileSync() to write student details into student.txt.
- Use fs.readFileSync() to read the file's contents back and print them to the console.
- Create calculator.js, define multiply(a, b) and divide(a, b), and export both using module.exports.
- Create app.js, import the calculator module using require('./calculator'), and call multiply() and divide() with sample values.
- Run app.js using Node.js and verify the multiplication and division results are printed.

6. PROGRAM

message.js

```
export function welcome(name) {  
    return `Welcome, ${name}!`;  
}
```

main.js

```
import { welcome } from './message.js';  
  
console.log(welcome("Aruna"));
```

system.js

```
const os = require('os');  
  
console.log("Operating System:", os.type());  
console.log("Platform:", os.platform());  
console.log("Architecture:", os.arch());  
console.log("Home Directory:", os.homedir());
```

filehandler.js

```
const fs = require('fs');  
  
fs.writeFileSync('student.txt', 'Name: Aruna\nBranch: AI & ML');  
  
const data = fs.readFileSync('student.txt', 'utf8');  
  
console.log(data);
```

calculator.js

```
function multiply(a, b) {  
    return a * b;  
}  
  
function divide(a, b) {  
    return a / b;  
}  
  
module.exports = {  
    multiply,  
    divide  
};
```

app.js

```
const calculator = require('./calculator');  
  
console.log("Multiplication:", calculator.multiply(6, 4));  
console.log("Division:", calculator.divide(20, 5));
```

7. CODE EXPLANATION

Code	Explanation
export function welcome()	Defines and exports a function from message.js using ES module syntax.
import { welcome } from './message.js'	Imports the welcome function into main.js using ES module syntax.
require('os')	Loads Node's built-in os module using CommonJS syntax.
os.type() / os.platform() / os.arch()	Return the operating system name, platform identifier and CPU architecture.
os.homedir()	Returns the path of the current user's home directory.
require('fs')	Loads Node's built-in fs (file system) module.
fs.writeFileSync(path, data)	Synchronously writes the given text to a file, creating or overwriting student.txt.
fs.readFileSync(path, 'utf8')	Synchronously reads the file's contents back as a text string.
module.exports = { multiply, divide }	Exports the multiply and divide functions from calculator.js using CommonJS syntax.
require('./calculator')	Imports the calculator module into app.js so its functions can be used.

8. TEST CASES AND EXECUTION DATA

Each script was executed independently using Node.js and its console output was verified.

Test Case	Action	Expected Result	Status
TC-01	Run node main.js	Console prints "Welcome, Aruna!", confirming the ES module import/export works	Pass
TC-02	Run node system.js	Console prints the Operating System, Platform, Architecture and Home Directory values	Pass
TC-03	Run the file-handling script	student.txt is created with the given text, then read back and printed to the console	Pass
TC-04	Run node app.js	Console prints "Multiplication: 24" and "Division: 4", confirming the CommonJS calculator module works	Pass

9. OUTPUT

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node main.js
Welcome, Aruna!
PS C:\Users\aruna\OneDrive\Desktop\fullstack> █
```

Output 1: Running node main.js - the ES module greeting is printed

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node system.js
Operating System: Windows_NT
Platform: win32
Architecture: x64
Home Directory: C:\Users\aruna
PS C:\Users\aruna\OneDrive\Desktop\fullstack> █
```

Output 2: Running node system.js - built-in os module details are printed

```
PS C:\Users\aruna\OneDrive\Desktop\fullstack>
PS C:\Users\aruna\OneDrive\Desktop\fullstack> node app.js
Multiplication: 24
Division: 4
PS C:\Users\aruna\OneDrive\Desktop\fullstack> █
```

Output 3: Running node app.js - the CommonJS calculator module results are printed

10. RESULT

Thus, custom modules were successfully created and used in Node.js with both ES module (import/export) and CommonJS (require/module.exports) syntax. Node's built-in os module was used to display system information, and the built-in fs module was used to write to and read from a file synchronously. The application was verified using multiple test cases, and all outputs matched the expected results.